Does the Routing Signal Matter in Dynamic Convolution?

A Capacity-Controlled Study on CIFAR-100

Ankit Kumar

Department of Mechatronics, University of Agder, Grimstad, Norway

11 September 2026

Abstract

Dynamic convolution replaces a single kernel with a bank of K kernels mixed by input-dependent weights. The literature credits its gains to input-adaptive kernel selection, but the routing signal itself has never been ablated against a capacity-matched control. We run that ablation across 90 training runs on CIFAR-100 and Tiny-ImageNet, separating three candidate mechanisms: the extra parameters of the bank, the linear re-parameterisation those parameters induce, and the routing itself. At the standard ResNet-18 width of 64, all three are null: a trained router, a router frozen at random initialisation, a router fed a mismatched input, a constant router, and a plain ResNet-18 with a quarter of the parameters all land within 0.25 pp of one another, and every arm is statistically equivalent to the plain network at a ±0.5 pp margin. Reducing width reveals why. At width 32 routing is worth 1.153 pp ($p < 10^{-4}$) and at width 16 0.532 pp ($p = 0.0023$), while the kernel bank on its own is worth nothing at every width tested. The effect tracks how far the network is from memorising its training set, and this yields a quantitative prediction we test on a held-out dataset: Tiny-ImageNet at width 32 trains to a loss seven times further from memorisation than CIFAR-100 does, and the routing effect there is 2.185 pp, 1.9 times larger. A sweep over $K \in \{2, 4, 8\}$ finds no pair of bank sizes distinguishable, so the benefit saturates at $K = 2$. Under matched parameter budgets, spending on width rather than on a kernel bank wins by 4.345 pp at 2.8M parameters and 2.313 pp at 11.2M, at between one third and one fifth of the training cost. All decision rules were fixed before the experiments ran.

1 Introduction

A dynamic convolution layer stores K kernels and combines them per input. Let $F \in \mathbb{R}^{B\times C\times H\times W}$ be the input feature map and $\{\mathcal{W}_k\}_{k=1}^{K}$ the kernel bank. The layer computes

$$\mathcal{W}_{\text{dyn}}(F) \;=\; \sum_{k=1}^{K} z_k(F)\,\mathcal{W}_k, \qquad \mathbf{z} = \operatorname{softmax}\!\left(\frac{g(\phi(F))}{\tau}\right), \tag{1}$$

where ϕ is a descriptor of the feature map, g a small multilayer perceptron, and τ a temperature. Nearly every published variant sets ϕ to global average pooling.

The field's stated explanation for why this helps is that $\mathbf{z}$ adapts the kernel to the input. That explanation has never been tested against the alternative that the gain comes from the K-fold increase in stored parameters, because published comparisons are against a network with K times fewer parameters. This paper supplies the missing controls.

Contributions.

1. A four-arm ablation that separates routing from capacity at fixed architecture, plus a plain-network reference (Section 2).
2. A capacity sweep across three widths showing the effect is real, non-monotone, and confined to the non-saturated regime (Section 5), together with a sweep over bank size K showing the benefit saturates at $K = 2$ (Section 7).
3. A held-out test of the resulting mechanism on Tiny-ImageNet, where the predicted direction and approximate magnitude both hold (Section 8).
4. Two parameter-matched comparisons showing that width dominates dynamic convolution at equal budget (Section 6).
5. A pre-registered protocol using equivalence testing rather than failure-to-reject, so null results carry evidential weight (Section 3).

2 Method

2.1 Routing signals

All arms share Eq. (1), the same backbone, the same optimiser and the same seeds. They differ only in how $\mathbf{z}$ is produced. Write $\mu(F) \in \mathbb{R}^C$ for the channel-wise global average.

`static` No dynamic layer. A plain ResNet-18, $K = 1$.

`uniform` $g(\cdot) \equiv \mathbf{0}$, hence $\mathbf{z} = (1/K, \ldots, 1/K)$ for every input. The bank exists and is trained; no routing occurs.

`gap` $\mathbf{z} = \mathrm{softmax}(g(\mu(F)))$ with g trained. The standard baseline.

`shuffled` $\mathbf{z}_i = \mathrm{softmax}(g(\mu(F)_{\pi(i)}))$ for a random permutation π redrawn each forward pass, at training *and* evaluation time. The marginal distribution of $\mathbf{z}$ matches `gap`; the correspondence between image and kernel is destroyed.

`frozen_random` Same architecture as `gap`, with every weight of g a buffer frozen at initialisation. Routing varies with the input but was never optimised.

2.2 Why `uniform` is the right capacity control

Proposition 1 (Collapse)**.** *When* $\mathbf{z} = (1/K, \ldots, 1/K)$ *identically, the layer computes* $F * \bar{\mathcal{W}}$ *with* $\bar{\mathcal{W}} = \frac{1}{K}\sum_k \mathcal{W}_k$*. Its effective parameter count equals that of a single static kernel.*

Proof. Immediate from Eq. (1): the mixture is constant, so $\mathcal{W}_{\text{dyn}} = \bar{\mathcal{W}}$ independent of F. □

Proposition 1 has a consequence worth stating plainly, because it changes what the arm measures. At inference `uniform` *is* a plain ResNet-18. It stores 44.18M parameters at width 64 but its effective count is 11.22M. It is therefore a control for *linear over-parameterisation* in the sense of Arora et al., not for representational capacity. Any difference between `uniform` and `static` is an optimisation effect, since the two span the same function class.

2.3 Two exact implementations

Convolution is linear in its weights, so

$$F * \Big(\sum_k z_k \mathcal{W}_k\Big) \;=\; \sum_k z_k\,(F * \mathcal{W}_k). \tag{2}$$

The left side (*combine*) builds a per-sample kernel and runs one grouped convolution, holding $B \cdot C_{\text{out}} \cdot C_{\text{in}} \cdot k^2$ floats. At width 64, batch 128, that is 1.2 GB per layer-4 convolution before activations. The right side (*blend*) runs K ordinary convolutions and mixes the outputs, costing $K\times$ the arithmetic and $K\times$ the activation memory. All experiments use *blend*. Equality of the two paths is asserted in the test suite to a tolerance of 10^{-10} in double precision across four routing signals.

2.4 Training protocol

Two datasets. CIFAR-100 (32×32, 100 classes) throughout, and Tiny-ImageNet (64×64, 200 classes) at width 32 only. The stem is deliberately not changed between them: keeping 3×3 stride 1 at 64×64 leaves the final feature map at 8×8 rather than 4×4 and costs roughly four times the compute in the deeper stages, but altering it would change the network between datasets and the comparison would no longer be like for like. Crop padding scales with resolution (4 at 32 pixels, 8 at 64), which is the convention for each.

ResNet-18 with a 3×3 stride-1 stem and no max-pool. All 3×3 convolutions inside residual blocks are replaced (16 dynamic layers); the stem and the 1×1 downsample shortcuts stay static. $K = 4$, $\tau = 1$. SGD with Nesterov momentum 0.9, learning rate 0.1, weight decay 5×10^{-4}, batch size 128, 5 warmup epochs then cosine decay over 150 epochs total. Augmentation is random crop with 4-pixel padding and horizontal flip, nothing further. Training uses bfloat16 autocast on a single RTX 4070 Ti. Six seeds, 0 through 5, shared across every arm. Base width $c \in \{16, 32, 64\}$.

3 Statistical protocol

3.1 Equivalence, not failure to reject

A large p from a difference test is not evidence of no effect. Since several of our claims are null claims, we use two one-sided tests (TOST) against a pre-registered margin $\delta = 0.5\,\text{pp}$. For paired differences d_i with mean $\bar{d}$ and standard error $s_{\bar{d}}$,

$$p_{\text{TOST}} = \max\Big\{1 - T_{n-1}\Big(\tfrac{\bar{d}+\delta}{s_{\bar{d}}}\Big),\; T_{n-1}\Big(\tfrac{\bar{d}-\delta}{s_{\bar{d}}}\Big)\Big\}, \tag{3}$$

and $p_{\text{TOST}} < 0.05$ is a positive assertion that the true difference lies inside $\pm\delta$. The margin was chosen because published dynamic-convolution gains at this scale are 1 to 3 pp, so equivalence within 0.5 pp contradicts them.

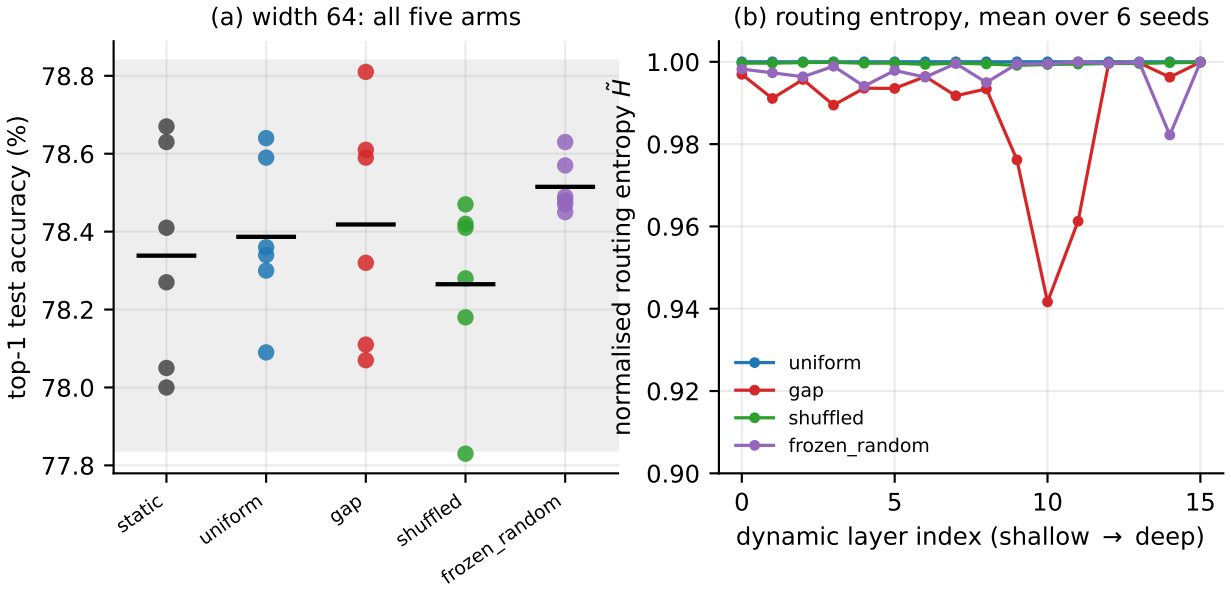


Figure 1: Width 64. (a) Every run of all five arms; the horizontal bar is the arm mean and the shaded band is the ±0.5 pp equivalence margin around `static`. (b) Normalised routing entropy $\tilde{H} = H(\mathbf{z})/\log K$ by layer depth, averaged over 6 seeds and the test set. `gap` does route ($\tilde{H}$ falls to 0.9289) and it changes nothing.

3.2 Why six seeds

The two-sided Wilcoxon signed-rank statistic on n pairs admits only 2^n equally likely sign assignments, so its attainable p is bounded below by 2^{1-n}: 0.25 at $n = 3$, 0.125 at $n = 4$, 0.0625 at $n = 5$, 0.031 at $n = 6$. Six is the smallest count that can reach $\alpha = 0.05$. We report Wilcoxon alongside the paired t-test throughout, and we report the bound wherever it binds.

3.3 Power

At the observed paired standard deviation of 0.257 pp for `gap` − `uniform` at width 64 with $n = 6$, the minimum detectable difference is 0.270 pp. Simulated power (2×10^5 draws) is 48.5 % at a true effect of 0.25 pp, 96.4 % at 0.5 pp and 100 % at 1.0 pp. The width-64 null is therefore not an artefact of low power: an effect of the size the literature reports could not have been missed.

3.4 Pre-registration

Arms, recipe, primary metric, seed count, margin and the decision table were committed before the first run finished. The primary metric is final-epoch top-1 accuracy rather than best-epoch, because best-epoch maximises over a noisy sequence and favours whichever arm caught a fortunate evaluation at high learning rate. No deviations were logged.

4 Results at the standard width

Table 3 and Figure 1 give the width-64 picture. Every arm is equivalent to a plain ResNet-18. The largest between-arm gap in means is 0.250 pp, against a median within-arm seed standard deviation of 0.220 pp: the arms differ by about as much as each arm differs from itself. A one-way ANOVA across the five arms gives $F(4, 25) = 0.954$, $p = 0.450$, $\eta^2 = 0.132$.

The `frozen_random` arm deserves emphasis. A router whose weights were never updated matches a trained one (0.097 pp, $p = 0.187$, $p_{\text{TOST}} = 0.0089$). So does `shuffled`, which routes each image using a different image's descriptor. And `gap` is not degenerate while doing it: Figure 1(b) shows normalised entropy falling to 0.9289 at layer 10, so the softmax is discriminating between kernels. It discriminates, and the discrimination does not matter.

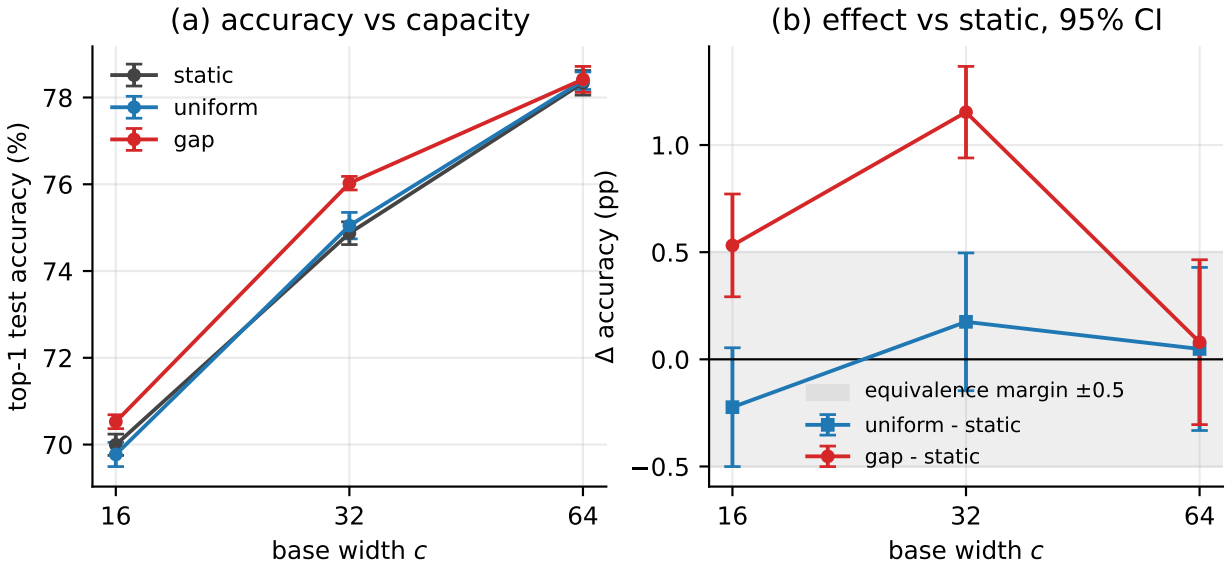


Figure 2: (a) Accuracy against base width; error bars are ±1 standard deviation over 6 seeds. (b) Paired difference from `static` with 95 % confidence intervals. The shaded band is the pre-registered equivalence margin.

5 The capacity sweep

Reducing width changes the answer. Table 3 and Figure 2 show `gap` beating `static` by 0.532 pp at width 16 ($p = 0.0023$) and 1.153 pp at width 32 ($p < 10^{-4}$), against 0.080 pp at width 64 ($p = 0.616$).

Two features of this result resist the obvious summary. First, it is **non-monotone**: the peak sits at width 32, not at the smallest width, so it cannot be described as an effect that grows steadily as capacity shrinks. Second, `uniform` remains equivalent to `static` at *all three* widths ($p_{\text{TOST}} = 0.0251, 0.0242, 0.0142$), so the kernel bank never contributes on its own and the linear over-parameterisation effect of Proposition 1 is absent throughout. Table 4 and Figure 4 separate the two.

5.1 Saturation explains the width-64 null

Figure 5 gives the mechanism. At width 64 every arm ends at a training loss of 0.008: the models memorise CIFAR-100 completely, and no architectural difference can express itself in test accuracy. At width 32 the loss is 0.017, still close to memorisation, and at width 16 it is 0.164, clearly outside that regime.

This also explains the non-monotonicity. At width 16, `gap` fits the training data far better than `static` (0.064 against 0.164) yet converts that into only 0.532 pp on test. The additional flexibility is being spent on fitting rather than generalising. Width 32 is where the flexibility is useful and the capacity is not yet sufficient to memorise, which is why the effect peaks there.

6 Matched parameter budgets

The sweep produces two near-exact parameter matches by accident, and they point the same way (Table 5, Figure 6). A plain network at width 32 (2.82M) beats `gap` at width 16 (2.79M) by 4.345 pp. A plain network at width 64 (11.22M) beats `gap` at width 32 (11.13M) by 2.313 pp. Both $p < 10^{-4}$, at two independent budgets.

Training cost compounds this. The bank costs 3.5× to 4.6× the wall-clock time of the static network at every width (Table 2). At 11.2M parameters, `gap` at width 32 takes 39.2 min per run to land 2.3 pp below a 18.8 min plain network.

7 Bank size

The width-32 peak was measured at $K = 4$. Since bank size interacts with capacity by construction, the peak might be an artefact of that choice. Table 9 sweeps $K \in \{2, 4, 8\}$ at width 32.

All three beat `static` by a similar margin, and Table 10 shows no pair of bank sizes is distinguishable: every adjacent comparison is equivalent at the ±0.5 pp margin, and $K = 8$ over $K = 2$ is 0.098 pp ($p = 0.409$). The routing benefit is saturated by $K = 2$, which is also the cheapest bank that admits routing at all.

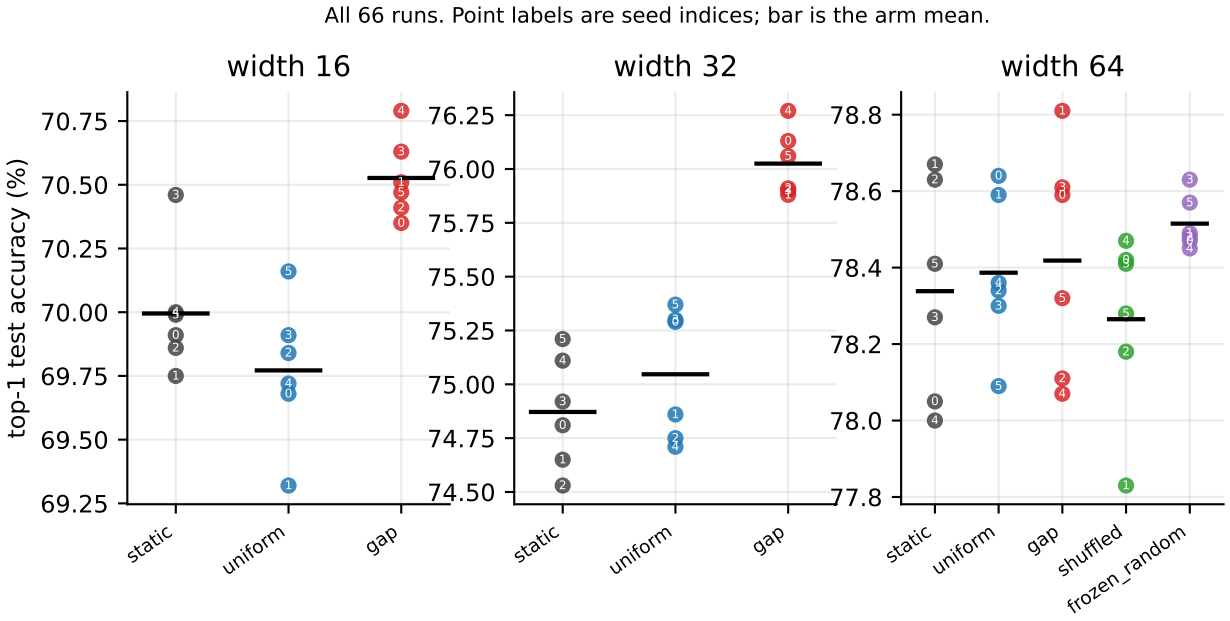


Figure 3: All 66 CIFAR-100 runs individually. Digits inside the markers are seed indices. Nothing is aggregated away.

The cost is not flat. Going from $K = 2$ to $K = 8$ multiplies stored parameters by 3.9 (5.64M to 22.12M) and training time by 2.5 (27.9 min to 69.3 min per run). Note also that the router itself barely grows (73.6K to 76.5K parameters), since only its output layer widens with K. All of the growth is the bank, and the bank is the component that contributes nothing.

8 A held-out test of the mechanism

Section 5 attributes the width-64 null to saturation. That is an explanation of results already in hand, so it needs a test that could have failed. The mechanism predicts that a dataset which leaves the network further from memorising its training set at the same width should show a larger routing effect.

Tiny-ImageNet at width 32 supplies that test. It has twice the training images over twice the classes at four times the pixels, and its static arm ends at a training loss of 0.1194 against CIFAR-100's 0.0170 at the same width, seven times further from the memorisation floor.

The prediction holds. Table 8 gives $\texttt{gap} - \texttt{static} = 2.185$ pp on Tiny-ImageNet (95 % CI $[+1.781, +2.589]$, $p < 10^{-4}$) against 1.153 pp on CIFAR-100, a factor of 1.9. The direction was predicted and the magnitude is of the predicted order.

What this does not establish is a capacity sweep on Tiny-ImageNet. One width and one bank size were run there, so the cross-dataset claim rests on a single point, and the inverted-U shape of Section 5 remains a CIFAR-100 result.

9 Tables

Table 1: Every CIFAR-100 run. Final-epoch top-1 test accuracy (%) for each of the 6 seeds, with arm mean and sample standard deviation.

width	arm	s0	s1	s2	s3	s4	s5	mean	sd
16	`static`	69.91	69.75	69.86	70.46	70.00	69.99	69.995	0.246
16	`uniform`	69.68	69.32	69.84	69.91	69.72	70.16	69.772	0.279
16	`gap`	70.35	70.51	70.41	70.63	70.79	70.47	70.527	0.160
32	`static`	74.81	74.65	74.53	74.92	75.11	75.21	74.872	0.262
32	`uniform`	75.29	74.86	74.75	75.30	74.71	75.37	75.047	0.305
32	`gap`	76.13	75.88	75.91	75.90	76.27	76.06	76.025	0.156
64	`static`	78.05	78.67	78.63	78.27	78.00	78.41	78.338	0.284
64	`uniform`	78.64	78.59	78.34	78.30	78.36	78.09	78.387	0.202
64	`gap`	78.59	78.81	78.11	78.61	78.07	78.32	78.418	0.299
64	`shuffled`	78.42	77.83	78.18	78.41	78.47	78.28	78.265	0.238
64	`frozen_random`	78.47	78.49	78.48	78.63	78.45	78.57	78.515	0.070

Table 2: Configuration and cost. Router parameters are a subset of the total. Training loss is the final-epoch mean over 6 seeds. Minutes are per run on one RTX 4070 Ti.

width	arm	stored params	router params	final train loss	min/run
16	`static`	707,684	0	0.1643	9.6
16	`uniform`	2,770,148	0	0.1653	11.4
16	`gap`	2,791,476	21,324	0.0638	33.7
32	`static`	2,818,404	0	0.0170	8.6
32	`uniform`	11,056,292	0	0.0170	12.9
32	`gap`	11,134,420	74,648	0.0132	39.2
64	`static`	11,220,132	0	0.0080	18.8
64	`uniform`	44,176,548	0	0.0081	25.2
64	`gap`	44,464,500	287,952	0.0074	86.0
64	`shuffled`	44,464,500	287,952	0.0083	92.3
64	`frozen_random`	44,176,548	0	0.0077	85.4

Table 3: Paired comparisons against `static`, $n = 6$ seeds. Wilcoxon cannot fall below 0.031 at $n = 6$; values at that bound are at the floor. Verdicts follow the pre-registered decision table.

width	arm	Δ (pp)	95 % CI	p_t	p_W	d_z	p_{TOST}	verdict
16	`uniform`	-0.223	[-0.500, +0.054]	0.0931	0.0938	-0.85	0.0251	equivalent
16	`gap`	+0.532	[+0.292, +0.771]	0.0023	0.0312	+2.33	0.6260	**beats**
32	`uniform`	+0.175	[-0.146, +0.496]	0.2206	0.3125	+0.57	0.0242	equivalent
32	`gap`	+1.153	[+0.940, +1.367]	$< 10^{-4}$	0.0312	+5.67	0.9997	**beats**
64	`uniform`	+0.048	[-0.332, +0.429]	0.7573	0.8438	+0.13	0.0142	equivalent
64	`gap`	+0.080	[-0.305, +0.465]	0.6158	0.5625	+0.22	0.0188	equivalent
64	`shuffled`	-0.073	[-0.602, +0.456]	0.7361	1.0000	-0.15	0.0464	equivalent
64	`frozen_random`	+0.177	[-0.121, +0.474]	0.1874	0.2188	+0.62	0.0191	equivalent

Table 4: Decomposition of the effect into the kernel bank and the routing.

	bank (`uniform−static`)		routing (`gap−uniform`)		total (`gap−static`)	
width	Δ (pp)	p	Δ (pp)	p	Δ (pp)	p
16	-0.223	0.0931	+0.755	0.0023	+0.532	0.0023
32	+0.175	0.2206	+0.978	0.0010	+1.153	$< 10^{-4}$
64	+0.048	0.7573	+0.032	0.7752	+0.080	0.6158

Table 5: Parameter-matched comparisons, paired by seed.

comparison	dynamic	static	Δ (pp)	95 % CI	p
`gap` @ w16 (2.79M) vs `static` @ w32 (2.82M)	70.527	74.872	+4.345	[+4.103, +4.587]	$< 10^{-4}$
`gap` @ w32 (11.13M) vs `static` @ w64 (11.22M)	76.025	78.338	+2.313	[+1.870, +2.757]	$< 10^{-4}$

Table 6: Routing behaviour at width 64. $\tilde{H} = H(\mathbf{z})/\log K$ is the normalised entropy of the mixing weights; spread is $\max_k z_k - \min_k z_k$ on the layer-mean weights. n is the number of layer-by-seed observations.

arm	mean $\tilde{H}$	min $\tilde{H}$	mean spread	max spread	n
`uniform`	1.0000	1.0000	0.0000	0.0000	96
`gap`	0.9885	0.9289	0.0690	0.2049	96
`shuffled`	0.9996	0.9986	0.0194	0.0405	96
`frozen_random`	0.9971	0.9774	0.0315	0.1069	96

Table 7: Every Tiny-ImageNet run at width 32, $K = 4$. Final-epoch top-1 test accuracy (%) per seed.

arm	s0	s1	s2	s3	s4	s5	mean	sd
`static`	61.93	61.61	61.57	61.46	61.27	61.52	61.560	0.217
`gap`	63.53	64.20	63.61	63.58	63.91	63.64	63.745	0.260

Table 8: The two datasets at width 32, $K = 4$. Training loss is the static arm's final-epoch mean. Minutes are static / gap per run.

dataset	static	gap	Δ (pp)	95 % CI	p	train loss	min/run
CIFAR-100	74.872	76.025	+1.153	[+0.940, +1.367]	$< 10^{-4}$	0.0170	8.6 / 39.2
Tiny-ImageNet	61.560	63.745	+2.185	[+1.781, +2.589]	$< 10^{-4}$	0.1194	54.2 / 284.2

Table 9: Bank size sweep, CIFAR-100 at width 32. Differences are against the `static` arm in the last row.

K	accuracy	sd	Δ (pp)	95 % CI	p	params (M)	min/run
2	75.950	0.148	+1.078	[+0.758, +1.399]	0.0003	5.64	27.9
4	76.025	0.156	+1.153	[+0.940, +1.367]	$< 10^{-4}$	11.13	39.2
8	76.048	0.242	+1.177	[+0.749, +1.604]	0.0009	22.12	69.3
static	74.872	0.262	–	–	–	2.82	8.6

Table 10: Pairwise comparisons between bank sizes. No pair is distinguishable, so the ordering of the means in Table 9 carries no information.

comparison	Δ (pp)	95 % CI	p	p_{TOST}	verdict
$K = 4$ vs $K = 2$	+0.075	[-0.087, +0.237]	0.2878	0.0005	equivalent
$K = 8$ vs $K = 4$	+0.023	[-0.286, +0.332]	0.8538	0.0053	equivalent
$K = 8$ vs $K = 2$	+0.098	[-0.182, +0.379]	0.4091	0.0072	equivalent

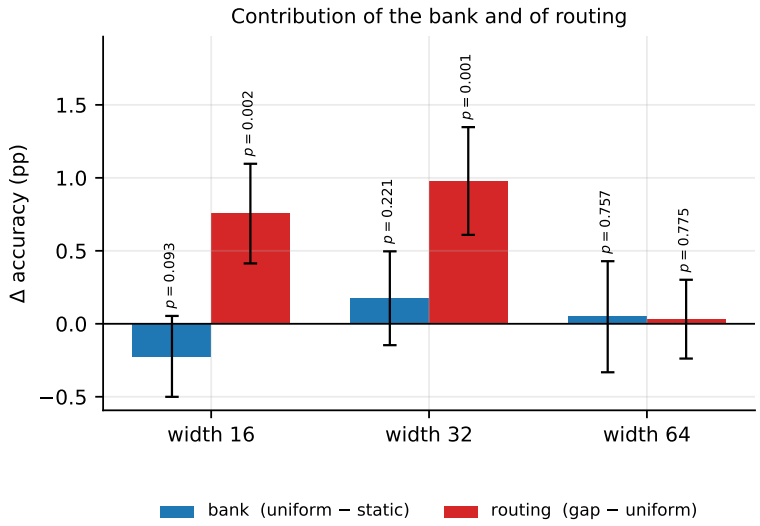


Figure 4: Decomposition. The bank alone (`uniform − static`) contributes nothing at any width. Routing (`gap − uniform`) carries the entire effect and vanishes at width 64. Error bars are 95 % CIs.

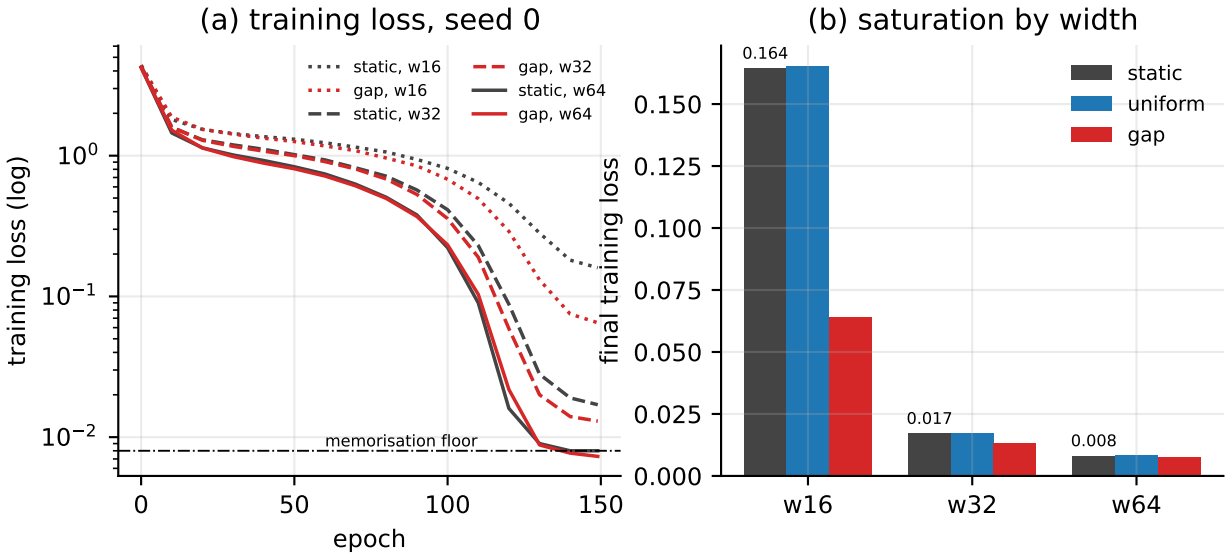


Figure 5: (a) Training loss for seed 0, log scale, sampled every 10 epochs. (b) Final training loss by width and arm. At width 64 every arm reaches 0.008, complete memorisation of the training set.

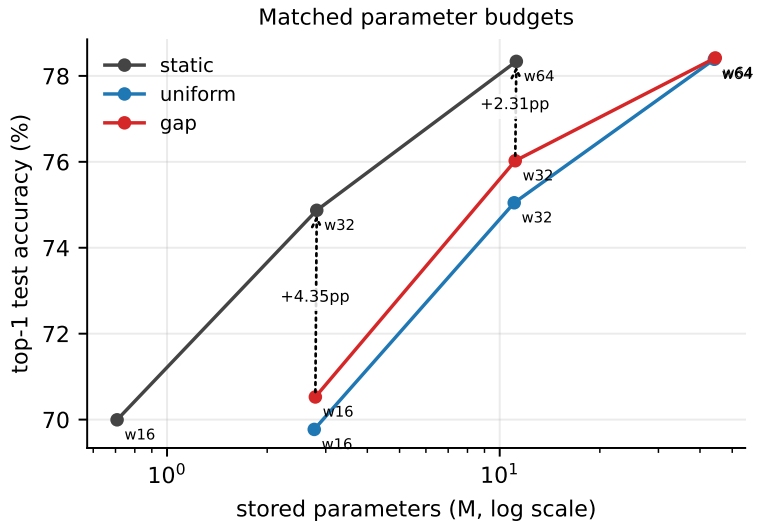


Figure 6: Accuracy against stored parameters. Dotted arrows join the two pairs matched to within 1 % on parameter count.

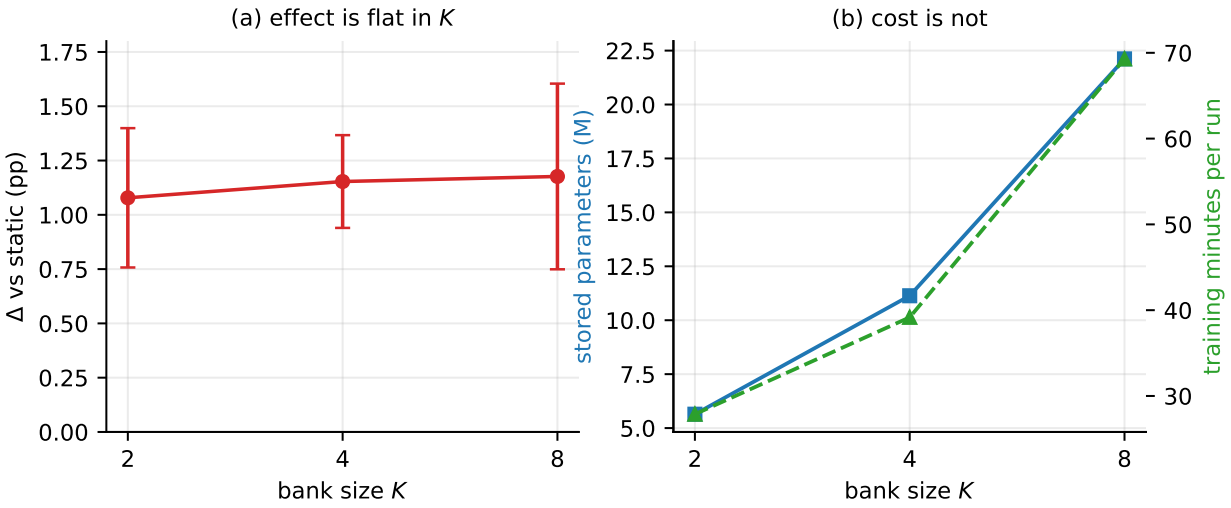


Figure 7: (a) Effect over `static` at width 32 for $K \in \{2, 4, 8\}$, 95 % CIs. (b) Stored parameters and training time over the same range. The benefit is flat; the cost is not.

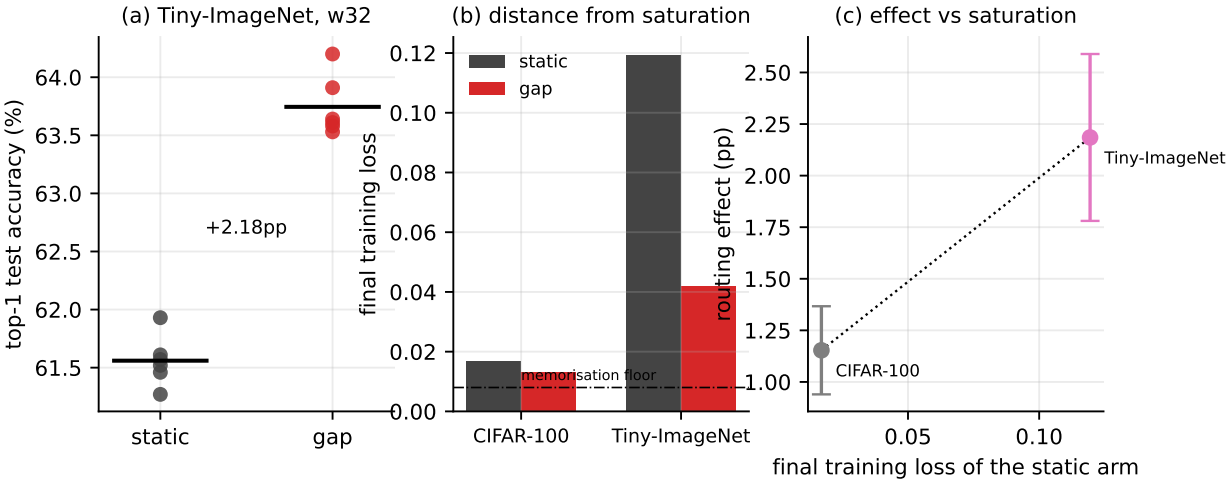


Figure 8: (a) All twelve Tiny-ImageNet runs at width 32. (b) Final training loss of each arm on both datasets; the dash-dotted line is the memorisation floor observed at width 64. (c) Routing effect against how far the static arm ends from memorisation.

10 Threats to validity

The capacity sweep is CIFAR-100 only. Tiny-ImageNet was run at one width and one bank size, so the cross-dataset test of Section 8 rests on a single point. The inverted-U shape across widths, and the claim that the bank contributes nothing at every width, are CIFAR-100 results. A second dataset agreeing at one width is weaker evidence than a second sweep would be.

Two datasets, both small. CIFAR-100 and Tiny-ImageNet are both low-resolution and fit on one consumer GPU. Whether the saturation mechanism survives at ImageNet scale is untested and is not testable on this hardware.

One architecture. ResNet-18 throughout. The bank size is swept in Section 7 and the width-32 peak survives it, but no other backbone was tried.

Global routing only. Every router here is spatially global. Spatially resolved variants such as DRConv and FDConv route on local signals and are not covered by these controls.

Effective capacity is not stored capacity. By Proposition 1, the `uniform` arm has the effective capacity of a plain network. Comparisons in this paper are explicit about which count is meant, but a reader taking stored parameters at face value would reach a different conclusion.

Provenance of the tabulated numbers. All width-64 records for `uniform`, `gap`, `shuffled` and `frozen_random` come from the result files written by the training script. The remaining arms were transcribed from the run logs, which report final accuracy verbatim; per-epoch curves for those arms are therefore available only at the 10-epoch sampling of the logs, which is what Figure 5(a) plots.

11 Conclusion

Dynamic convolution works, under conditions narrower than its literature suggests. The kernel bank contributes nothing on its own at any capacity tested, and no bank larger than $K = 2$ contributes anything further. The routing contributes, but only when the network cannot already memorise the data, and the contribution peaks in the middle of the capacity range rather than growing as capacity falls. At the standard ResNet-18 width on CIFAR-100 the entire construction is statistically equivalent to a plain network holding a quarter of the parameters.

That the effect tracks distance from memorisation is not only a description of the CIFAR-100 results. It predicted, before the runs were made, that Tiny-ImageNet at width 32 would show a larger effect, and the measured 2.185 pp against CIFAR-100's 1.153 pp is of the predicted direction and order.

The practical consequence is that at a fixed parameter budget the parameters are better spent on width. That held at both budgets we could match, by 2.3 pp and 4.3 pp, at a fraction of the training cost.

Reproducibility. 90 runs across two datasets, three widths, three bank sizes and 6 seeds, on one RTX 4070 Ti. Code, pre-registration, gate ledger with recorded evidence, and all result files accompany this report. Every table is generated from the result files rather than transcribed.